

ui.right_drawer 全面详细解析

`ui.right_drawer` 是 NiceGUI 框架中用于构建页面右侧抽屉式面板的核心布局组件，基于 Quasar 框架的 `Drawer` 组件封装，专注于页面右侧区域的功能扩展。该组件适用于右侧导航、详情面板、操作菜单、临时表单等场景，具备固定 / 滚动定位、视觉定制、状态联动等能力，且继承自 `Drawer`、`ValueElement`、`Element`、`Visibility` 基类，拥有丰富的属性与方法，可灵活适配各类页面布局需求。以下从核心特性、初始化参数、属性与方法、使用示例、注意事项等方面进行全面解析。

一、核心特性

- 布局定位**：固定在页面右侧，默认不随页面滚动（`fixed=True`），也可配置为随内容滚动，不遮挡页面主体核心区域。
- 视觉定制**：支持边框显示、阴影效果、边角扩展（顶部 / 底部与页面边角无缝衔接），可通过 `style`、`classes` 灵活调整背景色、宽度、内边距等样式。
- 状态可控**：支持默认展开 / 折叠、动态显示 / 隐藏、切换状态，初始展开状态可自动适配屏幕宽度（`value=None`），响应式表现优秀。
- 交互扩展**：可与按钮、表单、列表等组件深度集成，支持事件监听、数据绑定、动态资源添加，满足复杂交互场景（如右侧详情面板联动主体内容）。
- 层级分明**：支持通过 `elevated` 参数添加阴影，通过 `bordered` 参数添加边框，清晰区分抽屉与页面主体，提升视觉层次感。

二、初始化参数 (Initializer)

`ui.right_drawer` 的初始化参数用于配置组件基础行为与样式，所有参数均为可选，默认值已优化适配通用场景，详细说明如下：

参数名	类型	说明	默认值
value	bool None	初始展开状态：- <code>True</code> ：页面加载时默认展开- <code>False</code> ：页面加载时默认折叠- <code>None</code> ：根据屏幕宽度自动判断（大屏展开、小屏折叠），适配响应式布局	None
fixed	bool	定位方式控制：- <code>True</code> ：固定在右侧，滚动页面时保持位置不变- <code>False</code> ：随页面内容一起滚动，滚动到页面底部后不可见	True
bordered	bool	是否显示左侧边框（用于区分抽屉与页面主体），边框样式可通过 <code>style</code> 补充定制	False
elevated	bool	是否添加阴影效果，增强抽屉的视觉层级，使其突出于页面主体之上	False
top_corner	bool	是否向上扩展至页面顶部边角，消除顶部间隙，可与 <code>ui.header</code> 无缝衔接	False
bottom_corner	bool	是否向下扩展至页面底部边角，消除底部间隙，可与 <code>ui.footer</code> 无缝衔接	False

三、核心属性 (Properties)

继承自 `Drawer`、`ValueElement` 等基类，支持动态获取和修改组件状态与配置，核心属性如下：

属性名	类型	说明
classes	Classes[Self]	组件的 HTML 类名，支持通过 Tailwind/Quasar 类调整布局（如 <code>flex flex-col</code> ）和样式（如 <code>p-4</code> ），支持链式调用修改
client	Client	组件所属的客户端实例，关联 websocket 连接、布局上下文等，用于多客户端联动等高级场景
html_id	str	组件在 HTML DOM 中的唯一 ID（2.16.0 版本新增），可用于直接通过 DOM 操作组件
is_deleted	bool	组件是否已被删除（只读属性），用于判断组件生命周期状态
is_ignoring_events	bool	组件是否正在忽略事件（只读属性），用于事件控制逻辑
parent_slot	Slot None	组件的父插槽（可设置），用于调整组件在父容器中的插槽位置，适配复杂布局结构
props	Props[Self]	组件的 Quasar 原生属性，支持链式调用添加 / 删除（如 <code>props('backdrop rounded')</code> ），扩展组件原生能力
style	Style[Self]	组件的内联 CSS 样式，支持链式调用（如 <code>style('width: 300px').style('background-color: #f5f5f5')</code> ），精准控制组件外观
value	BindableProperty	组件当前展开 / 折叠状态（ <code>True</code> 为展开， <code>False</code> 为折叠），可绑定数据动态控制
visible	BindableProperty	组件的可见性（ <code>True</code> 为显示， <code>False</code> 为隐藏），可绑定数据动态控制

四、关键方法 (Methods)

提供丰富的方法用于组件操作、状态管理、事件绑定等，按功能分类解析核心方法：

1. 基础状态操作

方法名	参数	说明
show()	-	显示抽屉组件（仅控制可见性，不改变展开 / 折叠状态）
hide()	-	隐藏抽屉组件
toggle()	-	切换抽屉的展开 / 折叠状态（展开 ↔ 折叠），常用作按钮点击回调
delete()	-	删除抽屉组件及所有子元素，释放资源
clear()	-	清除抽屉内的所有子元素（保留抽屉本身，可重新添加内容）

方法名	参数	说明
update()	-	同步组件状态到客户端，修改样式、属性或子元素后需调用，确保变更生效
set_value()	value: Any	手动设置抽屉的展开 / 折叠状态（如 <code>set_value(True)</code> 强制展开）
set_visibility()	visible: bool	手动设置抽屉的可见性（如 <code>set_visibility(False)</code> 强制隐藏）

2. 样式与属性修改

方法名	核心参数	说明
default_classes()	add/remove/toggle/replace	批量修改同类组件的默认类名（需在组件实例化前调用，作用于所有 <code>ui.right_drawer</code> ）
default_props()	add/remove	批量修改同类组件的默认 Quasar 属性（需在实例化前调用，如全局添加边框）
default_style()	add/remove/replace	批量修改同类组件的默认 CSS 样式（需在实例化前调用，如全局设置默认宽度）
style()	css_str	设置内联样式，支持多轮链式调用，优先级高于 <code>classes</code>
classes()	class_str	设置 HTML 类名，可结合 Tailwind 实现快速布局（如 <code>classes('p-4 flex flex-col h-full')</code> ）
props()	props_str	设置 Quasar 原生属性，如 <code>props('backdrop')</code> 添加背景遮罩， <code>props('rounded')</code> 设置圆角

3. 数据绑定方法

支持将抽屉状态与 Python 对象属性动态绑定，实现单向 / 双向同步，简化状态管理：

方法名	核心参数	说明
bind_value()	target_object, target_name	双向绑定：抽屉展开状态与目标对象属性同步（一方修改，另一方自动更新）
bind_value_from()	target_object, target_name	单向绑定（从目标到组件）：目标对象属性变化时，同步更新抽屉展开状态
bind_value_to()	target_object, target_name	单向绑定（从组件到目标）：抽屉展开状态变化时，同步更新目标对象属性
bind_visibility()	target_object, target_name	双向绑定：抽屉可见性与目标对象属性同步
bind_visibility_from()	target_object, target_name	单向绑定（从目标到组件）：目标对象属性变化时，同步更新抽屉可见性

方法名	核心参数	说明
bind_visibility_to()	target_object, target_name	单向绑定（从组件到目标）：抽屉可见性变化时，同步更新目标对象属性

4. 事件绑定方法

方法名	核心参数	说明
on()	type, handler, js_handler	绑定组件事件（如 <code>click</code> 、 <code>update:model-value</code> 、 <code>click:backdrop</code> 等），支持 Python 处理器（服务端处理）或 JavaScript 处理器（客户端处理），2.18.0+ 版本可同时指定两者
on_value_change()	callback	绑定展开状态变化事件：当抽屉从展开变为折叠（或反之）时触发回调函数，回调参数包含事件详情（如 <code>e.value</code> 为新状态）

5. 其他实用方法

方法名	核心参数	说明
tooltip()	text	为抽屉添加悬浮提示文本（鼠标悬浮在抽屉边缘时显示）
mark()	*markers	为组件添加标记（如 <code>mark('right-panel filter')</code> ），用于测试查询或依赖管理
move()	target_container, target_index	将抽屉移动到其他父容器中，调整布局结构
add_resource()	path	为抽屉添加静态资源（如 CSS/JS 文件），扩展组件样式或功能
add_dynamic_resource()	name, function	为抽屉添加动态资源（通过函数返回资源响应），支持动态生成内容
get_computed_prop()	prop_name, timeout	异步获取组件的计算属性（需 <code>await</code> 调用），如获取抽屉实际宽度
run_method()	name, *args	在客户端执行组件原生方法（如 Quasar Drawer 的 <code>show()</code> 方法），需 <code>await</code> 调用

五、使用示例

1. 基础用法：右侧导航抽屉

```
from nicegui import ui

@ui.page('/basic_right_drawer')
def basic_drawer_demo():
    # 创建右侧抽屉：固定定位、带边框、阴影、上下无缝衔接
```

```

with ui.right_drawer(
    fixed=True,
    bordered=True,
    elevated=True,
    top_corner=True,
    bottom_corner=True,
    style='background-color: #ffffff; width: 260px'
) as right_drawer:
    # 抽屉标题
    ui.label('右侧导航').classes('text-lg font-bold p-4 border-b mb-4')
    # 导航菜单
    with ui.column().classes('p-2'):
        ui.button('首页', on_click=lambda: ui.notify('点击首页')).classes('w-full justify-start mb-2')
        ui.button('消息中心', on_click=lambda: ui.notify('点击消息中心')).classes('w-full justify-start mb-2')
        ui.button('个人设置', on_click=lambda: ui.notify('点击个人设置')).classes('w-full justify-start mb-2')
    # 底部关闭按钮
    ui.button('关闭抽屉', on_click=right_drawer.toggle).classes('mt-auto mx-4 mb-4 w-[calc(100%-2rem)]')

# 页面主体内容
with ui.column().classes('p-6'):
    ui.label('页面主体').classes('text-2xl font-bold mb-4')
    ui.button('打开右侧导航', on_click=right_drawer.toggle).props('elevated')
    [ui.label(f'主体内容行 {i}') for i in range(50)]

ui.run()

```

2. 高级用法：数据绑定与状态联动

```

from nicegui import ui

class AppState:
    def __init__(self):
        self.is_open = False # 控制抽屉展开状态
        self.is_visible = True # 控制抽屉可见性

@ui.page('/advanced_right_drawer')
def advanced_drawer_demo():
    state = AppState()

    # 创建抽屉并绑定状态
    right_drawer = ui.right_drawer(
        value=state.is_open,
        style='width: 280px; background-color: #f8f9fa'
    ).bind_value(state, 'is_open').bind_visibility(state, 'is_visible')

    with right_drawer:
        ui.label('动态抽屉演示').classes('text-lg font-bold p-4 border-b mb-4')
        # 绑定复选框与抽屉状态

```

```

ui.checkbox('自动展开抽屉', value=state.is_open).bind_value(state, 'is_open')
ui.checkbox('显示抽屉', value=state.is_visible).bind_value(state, 'is_visible')
# 监听状态变化并显示
ui.label(f'当前状态: {"展开" if state.is_open else "折叠"}').bind_text_from(
    state, 'is_open', lambda v: f'当前状态: {"展开" if v else "折叠"}'
)

# 页面主体控制区
with ui.row().classes('p-6'):
    ui.button('展开', on_click=lambda: right_drawer.set_value(True)).props('flat')
    ui.button('折叠', on_click=lambda: right_drawer.set_value(False)).props('flat')
    ui.button('切换展开/折叠', on_click=right_drawer.toggle).props('flat')
    ui.button('显示', on_click=right_drawer.show).props('flat')
    ui.button('隐藏', on_click=right_drawer.hide).props('flat')

# 监听抽屉状态变化事件
right_drawer.on_value_change(lambda e: ui.notify(f'抽屉状态变更为: {"展开" if e.value else "折叠"}'))

[ui.label(f'主体内容行 {i}') for i in range(50)]

ui.run()

```

3. 实用场景：右侧详情面板

```

from nicegui import ui

@ui.page('/detail_right_drawer')
def detail_drawer_demo():
    # 模拟商品数据
    products = [
        {'id': 1, 'name': '智能手表', 'price': 1999, 'desc': '全功能健康监测, 超长续航'},
        {'id': 2, 'name': '无线耳机', 'price': 899, 'desc': '主动降噪, 高清音质'},
        {'id': 3, 'name': '平板电脑', 'price': 2999, 'desc': '2K高清屏, 轻薄便携'}
    ]

    # 创建右侧详情抽屉（默认折叠）
    with ui.right_drawer(fixed=True, bordered=True, style='width: 320px') as detail_drawer:
        # 详情内容容器（动态更新）
        detail_container = ui.column().classes('p-4')
        empty_label = ui.label('请选择商品查看详情').classes('text-center text-gray-500 py-10')
        detail_container.add(empty_label)

    # 页面主体：商品列表
    with ui.column().classes('p-6'):
        ui.label('商品列表').classes('text-2xl font-bold mb-4')
        for product in products:
            # 商品卡片，点击显示详情
            with ui.card().classes('p-4 mb-4 w-full cursor-pointer') as card:
                card.on_click(lambda p=product: show_detail(p))
                ui.label(f'商品名称: {product["name"]}').classes('font-bold')
                ui.label(f'价格: ¥{product["price"]}')
```

```
# 动态更新详情面板内容
def show_detail(product):
    detail_container.clear() # 清空原有内容
    # 添加商品详情
    ui.label(f'商品详情').classes('text-xl font-bold mb-4', container=detail_container)
    ui.label(f'ID: {product["id"]}', container=detail_container)
    ui.label(f'名称: {product["name"]}', container=detail_container)
    ui.label(f'价格: ¥{product["price"]}', container=detail_container)
    ui.label(f'描述: {product["desc"]}', container=detail_container)
    ui.button('加入购物车', on_click=lambda: ui.notify(f'已添加 {product["name"]} 到购物车')).classes('mt-4', container=detail_container)
    detail_drawer.set_value(True) # 展开抽屉

ui.run()
```

六、注意事项

- 宽度与滚动控制**: 建议为抽屉设置固定宽度 (如 `width: 300px`) , 避免自适应宽度导致布局混乱; 当抽屉内容超出高度时, 可通过 `style('overflow-y: auto')` 开启纵向滚动, 提升用户体验。
- 固定定位与页面边距**: `fixed=True` 时, 抽屉可能遮挡页面主体右侧内容, 建议为页面主体添加右侧内边距 (如 `classes('pr-64')` , 对应抽屉宽度 256px) , 确保内容不被遮挡。
- 边角扩展适配**: `top_corner=True` 和 `bottom_corner=True` 需确保页面无默认 `body` 边距 (如 `margin: 0`) , 否则抽屉与页面边角会存在间隙, 可通过全局样式清除默认边距。
- 遮罩层使用**: 如需点击抽屉外部关闭抽屉, 可通过 `props('backdrop')` 添加背景遮罩, 再绑定 `on('click:backdrop', lambda: drawer.toggle())` 实现遮罩点击关闭逻辑。
- 事件节流优化**: 绑定高频事件 (如 `scroll`、`resize`) 时, 可通过 `on()` 方法的 `throttle` 参数设置节流时间 (单位: 秒) , 减少服务器与客户端的交互次数, 提升性能。
- 版本兼容性**: `html_id` 属性需 NiceGUI 2.16.0+ 版本, `js_handler` 支持同时指定 Python 与 JS 处理器需 2.18.0+ 版本, 使用时需确认项目依赖的 NiceGUI 版本匹配。
- 样式优先级**: `style` 方法设置的内联样式优先级高于 `classes` 中的类样式, 若需覆盖内联样式, 可在类中使用 `!important` 关键字 (如 `classes('width: 250px !important')`) 。

七、扩展参考

- 更多 Quasar 原生属性可通过 `props()` 方法传递, 详细支持的属性可参考 [Quasar Drawer 文档](#)。
- 样式定制可结合 Tailwind CSS 类 (如 `p-4`、`flex-col`、`mt-auto`) 实现复杂布局, 或通过 `style` 方法设置内联样式 (如 `background-color`、`box-shadow`) 。
- 若需实现响应式抽屉 (小屏幕自动隐藏, 大屏自动显示) , 可结合 `ui.screen` 组件监听屏幕尺寸变化, 动态修改 `visible` 或 `value` 属性。

ui.right_drawer

在 NiceGUI 中, `ui.right_drawer` (右侧抽屉) 是一种**浮动式的侧边容器组件**, 用于在页面右侧滑入 / 滑出额外的内容区域, 常被用于展示筛选条件、设置选项、详情信息等非核心内容, 既节省主页面空间, 又能按需展示交互内容。它基于 NiceGUI 的动态组件体系实现, 支持自定义尺寸、触发方式、显示状态和样式, 是提升页面交互体验的重要组件。

本文将从**基础使用**、**核心配置项**、**交互控制**、**样式自定义**和**实战场景示例**五个维度，详细阐述 `ui.right_drawer` 的使用方法。

一、基础使用

`ui.right_drawer` 是一个**上下文管理器组件**（需通过 `with` 语句嵌套子组件），默认处于**隐藏状态**，需通过触发条件（如按钮点击）控制其显示 / 隐藏。

1. 最简示例

```

from nicegui import ui

# 创建右侧抽屉（默认隐藏）
with ui.right_drawer() as drawer:
    ui.label('这是右侧抽屉的内容')
    ui.button('关闭抽屉', on_click=drawer.close)

# 触发按钮：打开抽屉
ui.button('打开右侧抽屉', on_click=drawer.open)

ui.run()
```

核心逻辑：

- 通过 `ui.right_drawer()` 创建抽屉实例并赋值给变量（如 `drawer`）。
- 调用实例的 `open()` 方法打开抽屉，`close()` 方法关闭抽屉。
- 抽屉内可嵌套任意 NiceGUI 组件（标签、按钮、表单、表格等）。

2. 自动销毁与持久化

默认情况下，抽屉关闭后**不会销毁内部组件**（再次打开时内容保留）；若需关闭时销毁组件，可设置 `persistent=False`：

```

# 非持久化抽屉：关闭后销毁内部组件，再次打开重新创建
with ui.right_drawer(persistent=False) as drawer:
    ui.label('非持久化抽屉，关闭后内容会销毁')
    # 每次打开都会重新生成随机数
    ui.label(f'随机数: {np.random.randint(1, 100)}') # 需导入 numpy

ui.button('打开抽屉', on_click=drawer.open)
```

二、核心配置项

`ui.right_drawer` 提供了丰富的参数来定制抽屉的行为和外观，核心配置项如下：

配置项	类型	默认值	说明
<code>value</code>	bool	<code>False</code>	初始显示状态： <code>True</code> 表示默认打开， <code>False</code> 表示默认隐藏
<code>persistent</code>	bool	<code>True</code>	是否持久化： <code>True</code> 关闭后保留组件， <code>False</code> 关闭后销毁组件

配置项	类型	默认值	说明
<code>bordered</code>	bool	<code>True</code>	是否显示抽屉与主页面的分隔边框
<code>width</code>	str/int	<code>300px</code>	抽屉宽度：支持像素（如 <code>300</code> 、 <code>'300px'</code> ）、百分比（如 <code>'20%'</code> ）、Flex 单位（如 <code>'1fr'</code> ）
<code>side</code>	str	<code>'right'</code>	抽屉位置：固定为 <code>'right'</code> （左侧抽屉用 <code>ui.left_drawer</code> ）
<code>on_open</code>	Callable	<code>None</code>	抽屉打开时触发的回调函数
<code>on_close</code>	Callable	<code>None</code>	抽屉关闭时触发的回调函数

配置项使用示例

```
from nicegui import ui

# 自定义配置的右侧抽屉
with ui.right_drawer(
    value=False, # 初始隐藏
    width='400px', # 宽度400px
    bordered=False, # 隐藏分隔边框
    on_open=lambda: ui.notify('抽屉已打开'), # 打开回调
    on_close=lambda: ui.notify('抽屉已关闭') # 关闭回调
) as drawer:
    ui.label('自定义宽度的右侧抽屉').classes('text-x1')
    ui.button('关闭', on_click=drawer.close).classes('mt-4')

ui.button('打开抽屉', on_click=drawer.open)

ui.run()
```

三、交互控制

`ui.right_drawer` 的核心交互是**显示 / 隐藏的动态控制**，除了直接调用 `open()` / `close()` 方法，还支持通过**绑定值**、**外部事件**和**快捷键**实现更灵活的交互。

1. 绑定值控制（双向数据绑定）

通过 NiceGUI 的**数据绑定**功能，将抽屉的显示状态与一个布尔值变量绑定，变量变化时抽屉自动显示 / 隐藏，反之亦然。

```
from nicegui import ui

# 创建布尔值变量，绑定到抽屉状态
drawer_visible = ui.value(False)

# 绑定 value 参数到变量
with ui.right_drawer(value=drawer_visible) as drawer:
    ui.label('通过绑定值控制的抽屉')
    # 点击按钮修改变量，间接关闭抽屉
    ui.button('关闭', on_click=lambda: drawer_visible.set(False))
```

```
# 按钮修改变量，间接打开抽屉
ui.button('打开抽屉', on_click=lambda: drawer_visible.set(True))

# 显示当前抽屉状态
ui.label('抽屉状态: ').bind_text_from(drawer_visible, '', lambda v: '打开' if v else '关闭')

ui.run()
```

2. 外部事件触发

可通过其他组件的事件（如选择框、表格行点击、定时器）触发抽屉的打开 / 关闭，示例如下：

```
from nicegui import ui
import time

with ui.right_drawer() as drawer:
    ui.label('通过定时器触发的抽屉')
    ui.button('关闭', on_click=drawer.close)

# 按钮点击后，3秒后自动打开抽屉
def open_after_3s():
    ui.notify('3秒后将打开抽屉...')
    time.sleep(3)
    drawer.open()

ui.button('3秒后自动打开抽屉', on_click=open_after_3s)

ui.run()
```

3. 点击遮罩层关闭

默认情况下，点击抽屉外部的**半透明遮罩层**会自动关闭抽屉，这是 NiceGUI 抽屉组件的内置行为，无需额外代码实现。若需禁用该行为，可通过自定义样式隐藏遮罩层（见下文样式自定义）。

四、样式自定义

`ui.right_drawer` 支持通过 `style()` 方法设置内联 CSS，或通过 `classes()` 方法使用 Tailwind CSS 类，自定义抽屉的背景、阴影、动画、遮罩层等样式。

1. 基础样式自定义

```
from nicegui import ui

with ui.right_drawer(
    width='350px',
    bordered=False
) as drawer:
    # 自定义抽屉内部样式：背景色、内边距、文字颜色
    drawer.style('''
        background: #f8f9fa;
        box-shadow: -5px 0 15px rgba(0, 0, 0, 0.1);
        padding: 20px;
```

```

'''
ui.label('自定义样式的右侧抽屉').classes('text-xl text-blue-600')
ui.divider() # 分隔线
ui.markdown('支持**Markdown**、表单、表格等任意组件')
ui.button('关闭', on_click=drawer.close).classes('mt-4 bg-red-500 text-white')

ui.button('打开抽屉', on_click=drawer.open).classes('bg-blue-500 text-white')

ui.run()

```

2. 隐藏遮罩层

若需禁用“点击遮罩层关闭抽屉”的行为，可通过 `ui.add_css()` 隐藏遮罩层：

```

from nicegui import ui

# 自定义CSS：隐藏抽屉遮罩层
ui.add_css('''
    .q-drawer__backdrop {
        display: none !important;
    }
''')

with ui.right_drawer() as drawer:
    ui.label('无遮罩层的抽屉，点击外部不会关闭')
    ui.button('关闭', on_click=drawer.close)

ui.button('打开抽屉', on_click=drawer.open)

ui.run()

```

3. 自定义动画

默认抽屉的滑入 / 滑出动画为从右到左，可通过 CSS 自定义动画效果（如淡入淡出、缩放）：

```

from nicegui import ui

# 自定义CSS：修改抽屉动画
ui.add_css('''
    /* 抽屉容器动画 */
    .q-drawer--right {
        animation: custom-slide-in 0.3s ease forwards !important;
    }
    .q-drawer--right.q-drawer--closed {
        animation: custom-slide-out 0.3s ease forwards !important;
    }
    /* 滑入动画 */
    @keyframes custom-slide-in {
        from { transform: translateX(100%); opacity: 0; }
        to { transform: translateX(0); opacity: 1; }
    }
    /* 滑出动画 */

```

```

        @keyframes custom-slide-out {
            from { transform: translateX(0); opacity: 1; }
            to { transform: translateX(100%); opacity: 0; }
        }
'''

with ui.right_drawer() as drawer:
    ui.label('自定义动画的抽屉')
    ui.button('关闭', on_click=drawer.close)

ui.button('打开抽屉', on_click=drawer.open)

ui.run()

```

五、实战场景示例

`ui.right_drawer` 在实际开发中常用于**筛选面板**、**详情展示**、**系统设置**等场景，以下是两个典型实战示例：

1. 数据筛选面板

在列表 / 表格页面中，用右侧抽屉作为筛选条件面板，筛选结果实时更新到主页面：

```

from nicegui import ui
import pandas as pd

# 模拟数据
data = pd.DataFrame({
    '姓名': ['张三', '李四', '王五', '赵六'],
    '年龄': [25, 30, 28, 35],
    '部门': ['研发', '产品', '研发', '销售']
})

# 存储筛选条件
dept_filter = ui.value('全部')
age_filter = ui.value(0)

# 创建表格组件
table = ui.table(
    columns=[{'name': col, 'label': col, 'field': col} for col in data.columns],
    rows=data.to_dict('records')
).classes('w-full')

# 右侧抽屉：筛选面板
with ui.right_drawer(width='350px') as drawer:
    ui.label('数据筛选').classes('text-xl font-bold mb-4')
    # 部门筛选
    ui.select(['全部', '研发', '产品', '销售'], label='部门', value='全部', on_change=lambda e:
dept_filter.set(e.value))
    # 年龄筛选
    ui.number(label='最小年龄', value=0, on_change=lambda e: age_filter.set(e.value))
    # 应用筛选按钮
    def apply_filter():
        filtered = data.copy()

```

```

        if dept_filter.value != '全部':
            filtered = filtered[filtered['部门'] == dept_filter.value]
            filtered = filtered[filtered['年龄'] >= age_filter.value]
            table.rows = filtered.to_dict('records')
            ui.notify('筛选条件已应用')

    ui.button('应用筛选', on_click=apply_filter).classes('mt-4 w-full bg-green-500 text-white')
    ui.button('关闭', on_click=drawer.close).classes('mt-2 w-full')

# 主页面按钮
ui.button('打开筛选面板', on_click=drawer.open).classes('mb-4')
ui.label('员工信息表').classes('text-2xl font-bold mb-2')

ui.run()

```

2. 详情展示面板

点击表格行或列表项时，在右侧抽屉中展示选中项的详细信息：

```

from nicegui import ui

# 模拟商品数据
products = [
    {'id': 1, 'name': '手机', 'price': 2999, 'desc': '6.7英寸大屏，5000mAh电池'},
    {'id': 2, 'name': '电脑', 'price': 5999, 'desc': '16GB内存，512GB固态硬盘'},
    {'id': 3, 'name': '平板', 'price': 1999, 'desc': '10.2英寸屏幕，支持手写笔'}
]

# 右侧抽屉：商品详情
with ui.right_drawer(width='400px') as drawer:
    detail_title = ui.label('商品详情').classes('text-xl font-bold')
    detail_content = ui.column().classes('mt-4')

# 渲染商品列表
def show_detail(product):
    # 清空之前的详情内容
    detail_content.clear()
    # 更新详情标题
    detail_title.set_text(f'商品详情: {product["name"]}')
    # 添加详情信息
    with detail_content:
        ui.label(f'商品ID: {product["id"]}')
        ui.label(f'价格: ¥{product["price"]}')
        ui.label(f'描述: {product["desc"]}')
    # 打开抽屉
    drawer.open()

ui.label('商品列表').classes('text-2xl font-bold mb-4')
for p in products:
    ui.button(p['name'], on_click=lambda p=p: show_detail(p)).classes('w-full mb-2')

```

```
ui.run()
```

六、总结

`ui.right_drawer` 是 NiceGUI 中功能强大的交互组件，其核心优势在于**按需展示内容**和**灵活的定制能力**：

1. **基础使用**：通过 `with` 语句嵌套子组件，调用 `open()` / `close()` 控制显示状态；
2. **核心配置**：支持自定义宽度、持久化、回调函数等，适配不同业务需求；
3. **交互控制**：通过数据绑定、外部事件实现动态触发，内置遮罩层关闭行为；
4. **样式定制**：结合 CSS/Tailwind 自定义背景、阴影、动画，提升视觉体验；
5. **实战场景**：适用于筛选面板、详情展示、系统设置等，是优化页面空间的重要工具。

在实际开发中，可根据业务需求结合其他组件（如表单、表格、图表），将 `ui.right_drawer` 打造成功能丰富的交互区域，大幅提升用户体验。